EconIRL: Estimators for Understanding and Predicting Dynamic Discrete Choices

Pranjal Rawat* John Rust†

April 6, 2026

Abstract

EconIRL implements twelve estimators for dynamic discrete choice models spanning structural econometrics (NFXP, CCP, MPEC) and inverse reinforcement learning (MCE-IRL, AIRL, GLADIUS). All estimators share a common interface and feed into unified post-estimation infrastructure for standard errors, identification diagnostics, and counterfactual simulation. We show that the IQ-Learn chi-squared objective is algebraically equivalent to a penalized conditional log-likelihood in the tabular DDC setting, connecting occupancy-matching IRL to structural maximum likelihood estimation. Three simulation studies validate the package: a tabular benchmark where all structural and IRL estimators converge to the same solution, a high-dimensional state space where sieve and neural methods maintain accuracy at lower computational cost, and a nonlinear reward environment where deep IRL outperforms linear models. The library is built on JAX for automatic differentiation and GPU acceleration and is MIT-licensed.

1 Introduction

Dynamic discrete choice models in economics and inverse reinforcement learning in computer science solve the same fundamental inference problem. An agent makes a sequence of decisions over time, and the researcher observes these decisions and wants to recover the reward function that rationalizes them. Rust (1987) formalized this problem as nested fixed-point maximum likelihood estimation, and the structural econometrics literature that followed developed a rich family of estimators including conditional choice probability methods and mathematical programming approaches (Aguirregabiria and Mira, 2010). Independently, Ziebart (2010) formalized the same problem as maximum causal entropy inverse reinforcement learning, and the machine learning literature developed its own family of estimators including adversarial methods and soft Q-learning. These two traditions share the same Bellman equation, the same discount factor, and the same logit choice kernel, yet they developed with different notation, different software ecosystems, and no unified pipeline for statistical inference or counterfactual analysis.

Existing software does not bridge these traditions. The `imitation` library (Gleave et al., 2022) implements several IRL algorithms but provides no standard errors, no identification diagnostics, and no counterfactual simulation. Stata packages for dynamic discrete choice cover NFXP and a handful of related estimators but cannot accommodate neural reward functions or adversarial

*PhD Candidate, Georgetown University. pp712@georgetown.edu
†Professor, Georgetown University. jf693@georgetown.edu

training. There is no library where a practitioner can specify a DDC model once, swap between structural and IRL estimators, and obtain valid inference from all of them through a single API. This gap forces applied researchers to maintain separate codebases for each estimator, making systematic comparison across methods unnecessarily difficult.

This paper introduces EconIRL, a Python library that provides twelve estimators for dynamic discrete choice models through a common `Estimator` protocol. Six estimators come from structural econometrics (NFXP, CCP, MPEC, NNES, SEES, TD-CCP) and six from inverse reinforcement learning (MCE-IRL, Deep MaxEnt, AIRL, GLADIUS, IQ-Learn, Behavioral Cloning). Every estimator returns an `EstimationSummary` object that provides standard errors by four methods (analytical, bootstrap, jackknife, and sandwich), identification diagnostics, and counterfactual simulation. The library is built on JAX (Bradbury et al., 2018) for automatic differentiation through fixed-point solvers via implicit differentiation, with Equinox (Kidger and Garcia, 2021) for neural network components. We also present a new theoretical result connecting the IQ-Learn chi-squared divergence objective to a penalized conditional log-likelihood in the tabular DDC setting (Rawat, 2026), which provides a formal bridge between occupancy-matching IRL and structural maximum likelihood estimation. Three simulation studies validate convergence, scaling, and nonlinear reward recovery across estimator families. The library is MIT licensed and pip-installable.

The remainder of the paper is organized as follows. Section 2 sets up notation and the DDC model. Section 3 describes the data pipeline from raw observations to estimation-ready panels. Section 4 presents each estimator with pseudocode. Section 5 covers post-estimation inference, diagnostics, and counterfactual analysis. Section 6 presents three simulation case studies and real-data applications. Section 7 concludes. Appendix A presents the IQ-Learn penalized MLE equivalence result.

2 Framework

The DDC model. An agent observes a discrete state $s_t \in \mathcal{S}$ at each period t, chooses an action $a_t \in \mathcal{A}$, and receives a flow payoff $u(s_t, a_t; \theta) + \sigma\, \epsilon_{t,a}$, where $\epsilon_{t,a}$ follows a Type I extreme value distribution. The state evolves according to a known transition kernel $P(s' \mid s, a)$, and the agent discounts future payoffs at rate $\beta \in [0, 1)$. This is the dynamic discrete choice framework introduced by Rust (1987). The agent's choice-specific value function satisfies the Bellman equation

$$Q(s, a; \theta) \;=\; u(s, a; \theta) \;+\; \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, a)\, V(s'), \tag{1}$$

where the integrated value function takes the log-sum-exp form implied by the extreme value error

$$V(s) \;=\; \sigma \log \sum_{a \in \mathcal{A}} \exp\bigl(Q(s, a; \theta)/\sigma\bigr). \tag{2}$$

The optimal policy is the softmax over choice-specific values

$$\pi(a \mid s; \theta) \;=\; \frac{\exp\bigl(Q(s, a; \theta)/\sigma\bigr)}{\sum_{a' \in \mathcal{A}} \exp\bigl(Q(s, a'; \theta)/\sigma\bigr)}. \tag{3}$$

This system is simultaneously the structural logit model of econometrics and the maximum causal entropy policy of Ziebart (2010). The structural side views Equations (1)–(3) as a likelihood model and estimates θ by maximum likelihood. The IRL side views them as a forward model mapping reward weights to behavior and estimates θ by matching observed and predicted feature expectations. The mathematical content is the same.

Identification. Rewards in this model are identified only up to additive state-dependent constants and global scale. For any bounded function $h\colon \mathcal{S} \to \mathbb{R}$, the shaped reward

$$r'(s,a) \;=\; r(s,a) \;+\; h(s) \;-\; \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a)\, h(s') \tag{4}$$

yields the same optimal policy as the original reward $r(s,a)$ (Ng et al., 1999; Magnac and Thesmar, 2002; Kim et al., 2021). This invariance means that raw reward magnitudes are not recoverable without further restrictions. The package resolves this ambiguity in three ways depending on the estimator family.

Table 1: Identification strategies across estimator families.

Strategy	Estimators	What is Recovered
Parametric $u = \theta^\top \phi(s,a)$	NFXP, CCP, MPEC, NNES, TD-CCP, SEES	θ (structural parameters)
Feature matching + norm	MCE-IRL	θ (reward weights)
Disentanglement	AIRL	$r(s)$ (state-only reward)

First, structural estimators restrict the flow utility to a linear-in-parameters form $u(s,a;\theta) = \theta^\top \phi(s,a)$, where ϕ is a known feature map. The parametric structure pins down the reward up to θ, and the likelihood identifies θ under standard rank conditions on the feature matrix. Second, MCE-IRL estimates reward weights θ by matching empirical and model-implied feature expectations, with a norm constraint or regularization penalty that resolves the scale ambiguity. Third, AIRL separates the discriminator into a state-only reward and a shaping potential, recovering $r(s)$ up to a constant by construction (Fu et al., 2018). Table 1 summarizes these three strategies.

Software abstractions. The library encodes the primitives above through three core types. `DDCProblem` is a frozen dataclass that stores the number of states $|\mathcal{S}|$, the number of actions $|\mathcal{A}|$, the discount factor β, and the scale parameter σ. Every estimator receives this object and uses it to construct value iteration operators and policy mappings. `Panel` is a list of `Trajectory` objects, where each trajectory records a sequence of state-action pairs (s_t, a_t) observed from a single agent. The panel structure preserves the serial dependence within trajectories while allowing estimators to treat agents as independent. The `Estimator` protocol requires every estimator to implement a single method, `.estimate(panel, utility, problem, transitions)`, and return an `EstimationSummary`. This API contract means that all twelve estimators are interchangeable parts. A user can swap NFXP for MCE-IRL or AIRL for CCP with one line of code and receive standard errors, identification diagnostics, and counterfactual predictions through the same post-estimation pipeline regardless of which estimator produced them.

3 Pre-Estimation

This section describes the data pipeline from raw observations to estimation-ready objects. The pipeline handles data loading, validation, state discretization, feature normalization, transition estimation, and utility specification.

Data types. Three core types encode the estimation problem. `DDCProblem` is a frozen dataclass that stores the MDP structure, including the number of states $|\mathcal{S}|$, the number of actions $|\mathcal{A}|$, the discount factor β, and the scale parameter σ of the extreme value shocks. `Trajectory` stores

a single individual's sequence of state-action-next-state tuples (s_t, a_t, s_{t+1}) as JAX arrays with matching lengths. `Panel` collects a list of trajectories and provides cached sufficient statistics via `.sufficient_stats()`, which computes state-action counts, empirical CCPs, transition frequencies, and the initial state distribution in a single pass over the data. Panels can be constructed from pandas DataFrames via `Panel.from_dataframe(df, state, action, id, next_state)` or saved and loaded as compressed NumPy archives via `.save_npz()` and `.load_npz()` for reproducibility.

Data loading. The `econirl.datasets` module provides loaders for real-world datasets spanning DDC applications and IRL trajectory data. DDC datasets include the Rust (1987) bus engine replacement data, Scania heavy truck component reliability, Keane-Wolpin career choices, and ICU sepsis treatment protocols. IRL trajectory datasets include Beijing taxi GPS traces via the T-Drive corpus, NYC Citibike station-to-station route choices, NGSIM highway lane-change trajectories, and Foursquare check-in sequences. Each loader returns a Panel object with metadata documenting the source, sample period, and variable definitions. For synthetic data, `simulate_panel(env, n_individuals, n_periods, seed)` generates panels from any `DDCEnvironment` with known ground truth parameters, enabling controlled parameter recovery experiments.

Data validation. Before estimation, `check_panel_structure()` validates the panel for missing values, unbalanced periods, and period gaps, returning a `PanelValidationResult` with error and warning lists. Four pre-estimation diagnostics are printed automatically before any `.estimate()` call. Feature matrix rank is computed on the flattened feature matrix of dimension $(|\mathcal{S}| \times |\mathcal{A}|) \times K$, and if the rank falls below K, estimation halts with an error. The condition number flags near-collinearity that inflates standard errors even when the matrix is technically full rank, with values above 10^6 triggering a warning. State coverage reports how many states have at least one observation. Single-action state counts identify states where only one action is ever observed, which produce degenerate CCP estimates.

State discretization and normalization. Continuous state variables are discretized via `discretize_state(valu method, n_bins)` using either equal-width or equal-count binning. The Rust (1987) convention of 5,000-mile bins is available via `discretize_mileage()`. Feature normalization during training uses `RunningNorm`, which implements the numerically stable online mean-variance algorithm of Chan, Golub, and LeVeque (1979) for features that span different scales. This is important when feature columns differ by orders of magnitude, as in the Rust bus model where operating cost is $O(10^{-3})$ and replacement cost is $O(1)$.

Transition estimation. When transition probabilities are not known from the model, they are estimated from data via `estimate_transition_probs()`, which counts state transitions from non-replacement observations and returns a probability vector that is expanded into the full $(|\mathcal{A}|, |\mathcal{S}|, |\mathcal{S}|)$ transition tensor. Group-stratified estimation is available via `estimate_transition_probs_by_group()` for heterogeneous transition dynamics across subpopulations such as bus groups or geographic regions. All estimators expect transitions in the shape $(|\mathcal{A}|, |\mathcal{S}|, |\mathcal{S}|)$, meaning `transitions[a, s, s']` gives $P(s'|s, a)$.

Utility specification. The `LinearUtility` class specifies $u(s, a; \theta) = \theta^\top \phi(s, a)$ from a feature matrix of shape $(|\mathcal{S}|, |\mathcal{A}|, K)$. The `anchor_action` parameter normalizes utility differences by subtracting the anchor action's features from all other actions, resolving the level non-identification discussed in Section 2. The class method `LinearUtility.from_environment(env)` extracts features

and parameter names automatically from any `DDCEnvironment`. For deep IRL estimators that learn nonlinear reward functions, `NeuralCost` provides a neural network parameterization of the reward.

4 Estimation

EconIRL implements twelve estimators organized into three families. The classical structural estimators (NFXP, CCP, MPEC) solve the forward problem exactly on tabular state spaces. The neural structural estimators (NNES, TD-CCP, SEES) approximate the value function to scale beyond tabular. The inverse reinforcement learning estimators (MCE-IRL, AIRL, GLADIUS, f-IRL, IQ-Learn) recover rewards from demonstrations, with behavioral cloning as a non-structural baseline. Table 2 summarizes the capability of each method.

Table 2: Estimator capabilities. All six classical and approximate estimators share the same parametric MLE objective and differ only in how they solve the Bellman constraint. The IRL estimators use fundamentally different identification strategies.

Estimator	Reference	Dir.	P req.	SEs	Scales	Transfer
NFXP-NK	Rust 1987	Fwd	✓	Analytical		
CCP-NPL	Hotz and Miller 1993	Fwd	✓	Hessian		
MPEC	Su and Judd 2012	Fwd	✓	Hessian		
NNES	Nguyen 2025	Fwd	✓	Valid	✓	
TD-CCP	Adusumilli and Eckardt 2025	Fwd		Robust	✓	
SEES	Luo and Sang 2024	Fwd	✓	Marginal	✓	
MCE-IRL	Ziebart 2010	Inv	✓	Bootstrap		
AIRL	Fu et al. 2018	Inv				✓
GLADIUS	Kang et al. 2025	Inv	✓	Projected	✓	
f-IRL	Ni et al. 2022	Inv	✓			
IQ-Learn	Garg et al. 2021	Inv	✓			
BC	Pomerleau 1991	Imit				

4.1 Classical Structural Estimators

NFXP-NK. The nested fixed point algorithm maximizes the exact log-likelihood $\ell(\theta) = \sum_i \log \pi(a_i|s_i;\theta)$ where choice probabilities come from solving the Bellman equation to machine precision at each outer step. The inner loop uses the SA-then-NK polyalgorithm of Iskhakov et al. (2016), which starts with successive approximation for global convergence and switches to Newton-Kantorovich near the fixed point for quadratic convergence. Gradients of the log-likelihood with respect to θ are computed analytically via the implicit function theorem, differentiating through the Bellman fixed point without backpropagation or finite differences. The outer loop uses BHHH optimization, which guarantees a positive-definite Hessian approximation from the outer product of per-observation score vectors. Algorithm 1 shows the main loop.

Algorithm 1: NFXP-NK (Rust, 1987; Iskhakov et al., 2016)

Input: Panel $\mathcal{D}$, features ϕ, transitions P, discount β
Initialize θ_0
for $k = 0, 1, 2, \ldots$ **do**
  Inner loop: Solve $V = T(V; \theta_k)$ via SA then NK
  Compute $Q(s, a) = u(s, a; \theta_k) + \beta \sum_{s'} P(s'|s, a)\, V(s')$
  Compute $\pi(a|s) = \text{softmax}(Q/\sigma)$
  Compute $\nabla_\theta \ell$ via implicit differentiation
  Update θ_{k+1} via BHHH or L-BFGS-B
  if $\|\nabla \ell\| < \varepsilon$ **then break**
end
Output: $\hat{\theta}$, V^*, π^*, $(-H)^{-1}$ for analytical SEs

CCP-NPL. The Hotz-Miller inversion lemma (Hotz and Miller, 1993) shows that under additive separability and IID extreme value shocks, observed choice probabilities $\hat{P}(a|s)$ uniquely invert to value differences via the emax correction $e(s, a) = \gamma_{\text{Euler}} - \log \hat{P}(a|s)$. This eliminates the inner Bellman loop entirely. Value recovery reduces to a single matrix inversion $(I - \beta F_\pi)^{-1}$, which costs $O(|\mathcal{S}|^3)$ once compared to thousands of value iterations in NFXP. Aguirregabiria and Mira (2002) show that iterating the Hotz-Miller inversion in CCP space (nested pseudo-likelihood, or NPL) converges to the MLE when the equilibrium is Lyapunov stable. Setting $K = 1$ gives a consistent but inefficient estimator. Setting $K = 5$ to 10 recovers full MLE efficiency. CCP-NPL is the only method in this package that extends naturally to dynamic games where computing Nash equilibria is infeasible.

MPEC. The mathematical programming with equilibrium constraints approach of Su and Judd (2012) treats the value function $V(s)$ as explicit decision variables alongside the structural parameters θ. The Bellman equation $V = T(V; \theta)$ becomes a set of equality constraints enforced via an augmented Lagrangian. This eliminates the inner loop entirely and jointly optimizes (θ, V) in a single L-BFGS-B call with increasing penalty on constraint violation. On the Rust bus benchmark at $\beta = 0.95$, MPEC achieves identical parameter estimates to NFXP while running six to nine times faster.

4.2 Neural Structural Estimators

NNES. The neural network estimation of structural models approach of Nguyen (2025) uses a two-phase procedure. In Phase 1 a neural network V_ϕ is trained to approximate the NPL Bellman target using data CCPs and the Hotz-Miller emax correction. In Phase 2 the learned V_ϕ is plugged into the CCP log-likelihood and the structural parameters θ are optimized via L-BFGS-B. The key theoretical contribution is the zero Jacobian property (Proposition 1): at the true CCP, $\partial \phi_{\theta_0}(P^*, V^*)/\partial P = 0$, so neural network errors in V have only second-order effects on the likelihood. This Neyman orthogonality guarantees that $\hat{\theta}$ is $\sqrt{n}$-consistent even when V_ϕ converges at the slower rate $o_p(n^{-1/4})$, and the estimator achieves the semiparametric efficiency bound without bias correction or cross-fitting.

TD-CCP. The temporal-difference CCP estimator of Adusumilli and Eckardt (2025) is the only method in this package that does not require known or estimated transition densities. The key insight is that the CCP value function components $h(a, x)$ and $g(a, x)$ can be characterized via

temporal-difference fixed-point equations that depend only on sample expectations over observed transitions (s, a, s'), not on the density $p(s'|s, a)$. Algorithm 2 shows the linear semi-gradient variant, which solves for h via a single matrix inversion.

Algorithm 2: TD-CCP, linear semi-gradient (Adusumilli and Eckardt, 2025)

Input: Panel $\mathcal{D}$ with transitions (s_t, a_t, s_{t+1}), features ϕ, basis Ψ
Estimate CCPs $\hat{P}(a|s)$ from data frequencies
for *each action a and feature k* **do**
  Solve $(I - \beta \hat{F}_\pi)\, h_{a,k} = \phi_{a,k}$ via matrix inversion
end
Construct pseudo-outcome $Y_i = \sum_k \theta_k\, h_k(a_i, x_i)$
Minimize pseudo-likelihood over θ via GMM
Compute robust SEs via locally debiased moment condition
Output: $\hat{\theta}$, robust SEs valid at $\sqrt{n}$-rate

The locally robust moment condition (Theorem 5 of Adusumilli and Eckardt 2025) uses a backward value function λ and debiased moment ζ to correct for first-stage estimation error, producing standard errors valid at parametric rates despite nonparametric first stages. The per-feature $h(a, x)$ decomposition also provides interpretable diagnostics showing how each feature contributes to the value function.

SEES. The sieve estimator of Luo and Sang (2024) approximates $V(s) \approx \Psi(s)\,\alpha$ where Ψ is a deterministic sieve basis of Fourier or polynomial functions. Joint L-BFGS-B optimization over (θ, α) with an ℓ^2 penalty on α enforces Bellman consistency without ever solving a fixed point. The basis matrix is deterministic and continuation values reduce to a single matrix multiply $P_a \Psi$, making the computational cost $O(\text{basis_dim})$ independent of $|\mathcal{S}|$. On the Rust bus benchmark, SEES achieves parameter estimates within one percent of NFXP while running fifteen to thirty-four times faster. No neural network training is involved.

4.3 Inverse Reinforcement Learning Estimators

MCE-IRL. Maximum causal entropy IRL (Ziebart, 2010) is the bridge estimator connecting the econometrics and machine learning traditions. The backward pass runs soft value iteration to compute V, Q, and π from the current reward weights θ. The forward pass computes state visitation frequencies D_π via the occupancy measure fixed point $(I - \beta P_\pi^\top)^{-1} \rho_0$. The gradient is the feature matching residual $\nabla_\theta \mathcal{L} = \mathbb{E}_{\text{data}}[\phi] - \mathbb{E}_\pi[\phi]$. Algorithm 3 shows the main loop.

Algorithm 3: MCE-IRL (Ziebart, 2010)

Input: Panel $\mathcal{D}$, features ϕ, transitions P, discount β
Compute empirical features $\bar{f} = \frac{1}{N}\sum_i \phi(s_i, a_i)$
Initialize θ_0
for $k = 0, 1, 2, \ldots$ **do**
  Backward: Soft VI $\to V$, Q, π from θ_k
  Forward: $D_\pi = (I - \beta P_\pi^\top)^{-1}\rho_0$
  $\bar{f}_\pi = \sum_s D_\pi(s) \sum_a \pi(a|s)\,\phi(s,a)$
  $g = \bar{f} - \bar{f}_\pi$ `// feature matching residual`
  $\theta_{k+1} = \theta_k + \alpha\, g$
  if $\|g\| < \varepsilon$ ***and*** $\|D_\pi - D_{data}\|_\infty < \tau$ **then break**
end
Output: $\hat{\theta}$ (reward weights), bootstrap SEs

This gradient is simultaneously the MCE gradient of Ziebart (2010) and the score of the DDC conditional logit likelihood. An economist sees structural logit with feature matching. A machine learning researcher sees maximum entropy IRL. They solve the same optimization problem. The deep variant replaces $\theta^\top\phi(s,a)$ with a neural network $r_\psi(s,a)$ following Wulfmeier et al. (2016), gaining flexibility at the cost of interpretability and standard errors.

AIRL. Adversarial inverse reinforcement learning (Fu et al., 2018) is the only IRL method in this package with a proven transfer guarantee. The discriminator takes the form $D(s,a,s') = \exp(f)/(\exp(f) + \pi(a|s))$ where $f(s,a,s') = g_\psi(s) + \beta\, h_\psi(s') - h_\psi(s)$. This structure disentangles the state-only reward $g(s)$ from the shaping potential $h(s)$. Algorithm 4 shows the adversarial training loop.

Algorithm 4: AIRL (Fu et al., 2018)

Input: Expert demonstrations $\mathcal{D}_E$, transitions P, discount β
Initialize reward g_ψ, shaping h_ψ, policy π_ω
for $k = 0, 1, 2, \ldots$ **do**
  $f(s,a,s') = g_\psi(s) + \beta\, h_\psi(s') - h_\psi(s)$
  $D(s,a,s') = \exp(f) \,/\, \big(\exp(f) + \pi_\omega(a|s)\big)$
  Update ψ: $\max_\psi\ \mathbb{E}_{\mathcal{D}_E}[\log D] + \mathbb{E}_\pi[\log(1-D)]$
  Update ω: policy improvement against f
  if *converged* **then break**
end
Output: $g_\psi(s)$ (disentangled reward, transfers across environments)

Theorem 5.1 of Fu et al. (2018) proves that the state-only reward recovered by AIRL is disentangled with respect to all dynamics, meaning it transfers across environments with different transition functions. This is the method to use when rewards learned in simulation must be deployed under different real-world dynamics.

GLADIUS. The ERM-IRL estimator of Kang et al. (2025) parameterizes $Q(s,a)$ and $\zeta(s,a) = \mathbb{E}[V(s')|s,a]$ with neural networks and trains them with alternating mini-batch SGD. Even-numbered batches update ζ via MSE against $\log\sum\exp(Q)$. Odd-numbered batches update Q via the negative log-likelihood. After convergence, structural parameters are recovered via action-difference projection $\hat{\theta} = (\Delta\Phi^\top\Delta\Phi)^{-1}\Delta\Phi^\top\Delta Q$ where Δ denotes differences from a reference action, eliminating the unidentified additive constant. A known limitation is that Q is identified only up to a state-dependent

constant $c(s)$ that leaks through asymmetric transitions, producing systematic bias in the operating cost parameter on the Rust bus benchmark.

f-IRL. The f-IRL estimator of Ni et al. (2022) minimizes the f-divergence $D_f(\rho_E \| \rho_\pi)$ between expert and policy state-action occupancy measures over a tabular reward function. The choice of f provides a menu of robustness properties. KL divergence gives maximum likelihood equivalence. Chi-squared divergence gives variance sensitivity. Total variation gives robustness to outlier demonstrations. No features need to be specified.

IQ-Learn. The IQ-Learn estimator of Garg et al. (2021) collapses the min-max IRL problem to a concave maximization over Q alone, since the optimal policy is deterministically the softmax of Q. Reward is recovered via the inverse Bellman operator $r(s,a) = Q(s,a) - \beta \sum_{s'} V(s')\, p(s'|s,a)$. Appendix A shows that in the tabular DDC setting this objective is algebraically equivalent to a penalized conditional log-likelihood.

BC (baseline). Behavioral cloning (Pomerleau, 1991) estimates the policy directly via frequency counting $\hat{P}(a|s) = N(s,a)/N(s)$. No transitions, no Bellman equation, no reward function, and no value function are involved. Ross et al. (2011) show that BC error compounds quadratically under distribution shift at rate $O(T^2\varepsilon)$. Any IRL method that cannot outperform BC has not learned from MDP structure. BC establishes the floor for every benchmark in this paper.

5 Post-Estimation

Every estimator in EconIRL returns the same `EstimationSummary` object, which feeds into a shared infrastructure for inference, diagnostics, goodness-of-fit evaluation, and counterfactual simulation. This section describes each component.

Standard errors. EconIRL provides four methods for computing standard errors, selected via the `se_method` parameter on `.estimate()`. Asymptotic standard errors use the inverse of the negative Hessian, $\mathrm{Var}(\hat{\theta}) = (-H)^{-1}$, which is valid under correct specification of the likelihood. Robust sandwich standard errors compute $H^{-1}BH^{-1}$ where $B = \sum_i g_i g_i^\top$ is the outer product of per-observation score vectors, accounting for potential misspecification without requiring a correctly specified variance. Bootstrap standard errors resample individuals with replacement via `TrajectoryPanel.resample_individuals()` and re-estimate on each sample. The warm-start one-step NPL bootstrap runs only one to three Newton steps from the MLE estimate on each bootstrap sample, achieving quadratic convergence with roughly 100x speedup over full re-estimation. Clustered standard errors aggregate the score vectors within individuals before forming the outer product, with a small-sample correction factor of $G/(G-1) \times (N-1)/(N-K)$ where G is the number of clusters, N is the number of observations, and K is the number of parameters. The Hessian is computed analytically for NFXP via implicit differentiation through the Bellman fixed point (Iskhakov et al., 2016) and numerically via central finite differences for all other estimators.

Identification diagnostics. Post-estimation, the library performs eigenvalue analysis of the negative Hessian at the optimum. The `check_identification()` function classifies the result into one of four categories based on eigenvalue thresholds. If the rank of $-H$ is less than K, the model is under-identified. If any eigenvalue is non-positive, the optimum is a saddle point or local minimum. If the condition number exceeds 10^6, the model is weakly identified. Otherwise the

model is well-identified. Parameter correlations exceeding 0.95 are flagged as a warning of near-collinearity (Magnac and Thesmar, 2002). Parameter sensitivity analysis computes the elasticity of each parameter estimate with respect to all other parameters, revealing which parameters are tightly coupled and which are independently identified by the data.

Goodness of fit. The `GoodnessOfFit` object reports log-likelihood, AIC, BIC, McFadden pseudo-R^2, and prediction accuracy, defined as the fraction of observations where the modal predicted action matches the observed action. Four additional fit metrics are available. The Brier score measures calibration of the predicted choice probabilities, with values ranging from 0 (perfect) to 2 (worst possible). The KL divergence between data and model CCPs, weighted by empirical state frequencies, measures the information loss from using the model distribution in place of the empirical distribution. Efron pseudo-R^2 provides a variance-ratio analog of the linear R^2 for discrete choice settings. A Pearson χ^2 CCP consistency test compares the observed and predicted choice frequencies at each state, returning a test statistic, degrees of freedom, and p-value for the null hypothesis that the model generates the data.

EstimationSummary. The `EstimationSummary` object provides parameter estimates, standard errors, t-statistics, p-values, and 95 percent confidence intervals. Joint linear hypotheses $H_0\colon R\theta = r$ are tested via Wald statistics with associated degrees of freedom and p-values. The `.summary()` method prints a formatted table in the style of the `statsmodels` library, with one row per parameter. The `.to_latex()` method produces a publication-ready `booktabs` table. The `.to_dataframe()` method returns a pandas DataFrame for programmatic analysis. This uniform output format means that switching from NFXP to MCE-IRL or from CCP to AIRL requires changing only the estimator class, with no changes to downstream inference or reporting code.

Counterfactual simulation. The library supports four types of counterfactual analysis, following the taxonomy of Aguirregabiria and Mira (2010). Type 1 (state extrapolation) evaluates the estimated policy at shifted state indices without re-solving the Bellman equation, answering questions about what happens when the initial state distribution changes. Type 2 (environment change) re-solves the Bellman equation under modified transition dynamics while holding the reward function fixed, answering questions like "what if component reliability improves by 20 percent." Type 3 (reward change) re-solves under modified utility parameters while holding transitions fixed, answering "what if replacement cost doubles." Type 4 (welfare decomposition) decomposes the total welfare change into contributions from each channel. Elasticity analysis sweeps a single parameter over a user-specified range, re-solves for the optimal policy at each point, and reports the policy response surface. The `simulate_counterfactual()` function generates synthetic panels under both baseline and counterfactual policies, producing action frequency and state frequency comparisons for direct visualization. Stationary distributions under each policy are computed via power iteration to assess long-run behavioral effects.

6 Simulation Studies and Applications

We validate EconIRL with three simulation studies designed to test distinct aspects of the estimator portfolio. Case Study 1 uses a small tabular MDP to verify that all estimators converge to the correct solution. Case Study 2 uses a high-dimensional state space to test computational scaling. Case Study 3 uses a nonlinear reward environment to demonstrate where deep IRL outperforms linear models. We close with real-data applications.

6.1 Case Study 1: Tabular Convergence

We simulate data from the Rust (1987) bus engine replacement model with 90 mileage states, 2 actions (keep, replace), discount factor $\beta = 0.9999$, and true parameters $\theta_{\text{oc}} = 0.001$ (operating cost) and $\theta_{\text{rc}} = 3.0$ (replacement cost). We generate 200 individuals observed for 100 periods each, producing 20,000 observations.

Table 3: Case Study 1: Rust bus engine benchmark. True parameters: $\theta_{\text{oc}} = 0.001$, $\theta_{\text{rc}} = 3.0$. Data: 200 individuals $\times$ 100 periods, $\beta = 0.9999$.

	Estimator	$\hat{\theta}_{\text{oc}}$	$\hat{\theta}_{\text{rc}}$	Log-Lik	Cosine	Time (s)
Classical	NFXP-NK	0.0012	3.011	−4263.2	1.0000	0.1
	CCP-NPL	0.0012	3.011	−4263.2	1.0000	0.1
	MCE-IRL	0.0012	3.011	−4263.2	1.0000	0.1
Approx.	NNES	0.0315	3.073	−4264.0	0.9999	16.0
	TD-CCP	0.0011	2.943	−4265.0	1.0000	129.6
	SEES	0.0095	2.929	−4268.3	1.0000	1.5
	GLADIUS	0.0015	2.367	−4264.5	1.0000	10.4
IRL	AIRL	0.0103	3.897	−4467.1	1.0000	607.7
	IQ-Learn	−0.066	1.781	−4678.1	0.9993	<1
	BC	—	—	−4238.1	—	<1

The first three rows of Table 3 confirm the central theoretical result. NFXP, CCP, and MCE-IRL achieve identical log-likelihood (−4263.2) and cosine similarity (1.0000) to the true parameters. These three estimators solve the same maximum likelihood problem through different computational strategies and arrive at the same answer. This is the bridge between structural econometrics and maximum causal entropy IRL. MCE-IRL conditions each action only on information available at decision time, which is exactly the causal entropy of Ziebart (2010) and what economists call rational expectations in sequential decisions.

The approximate estimators (NNES, TD-CCP, SEES, GLADIUS) all achieve cosine similarity above 0.9999, confirming that their value function approximations are accurate enough for parameter recovery. SEES runs in 1.5 seconds compared to 16 seconds for NNES and 130 seconds for TD-CCP, because its sieve basis avoids neural network training entirely. GLADIUS recovers the replacement cost within 21 percent but underestimates it due to the state-dependent identification constant that leaks through asymmetric transitions. This is a structural bias, not a tuning problem.

IQ-Learn achieves lower log-likelihood than NFXP (−4678 versus −4263) because its ℓ^2 reward penalty trades fit for a smaller implied reward, as predicted by the penalized MLE equivalence in Appendix A. AIRL is the slowest method at 608 seconds due to adversarial training but is the only estimator with a proven transfer guarantee across environments.

6.2 Case Study 2: High-Dimensional State Space

We use the multi-component bus environment with $K = 2$ independent components and $M = 20$ mileage bins per component, producing $20^2 = 400$ states. True parameters are replacement cost 3.0, operating cost 0.001, and quadratic cost 0.0005. We simulate 500 individuals for 100 periods (50,000 observations) and compare NFXP against SEES with varying basis dimensions and NNES.

Table 4: Case Study 2: Multi-component bus (400 states, $\beta = 0.99$). SEES basis sweep compared to NFXP. RMSE is computed against NFXP parameter estimates.

Estimator	$\hat{\theta}_{\text{rc}}$	RMSE vs NFXP	Log-Lik	Time (s)
NFXP (exact)	3.052	—	−9685.0	7.6
SEES ($K=4$)	3.031	0.039	−9685.3	1.6
SEES ($K=8$)	3.076	0.044	−9684.3	1.3
SEES ($K=12$)	3.073	0.042	−9684.0	2.0
SEES ($K=16$)	3.075	0.043	−9684.0	2.2
SEES ($K=20$)	3.071	0.040	−9683.4	2.5
NNES (neural)	3.525	—	−9860.3	24.6

SEES with as few as four basis functions recovers the replacement cost within one percent of the NFXP estimate at roughly four times the speed. Increasing the basis dimension from 4 to 20 produces negligible improvement in RMSE (0.039 to 0.040), confirming that the value function is smooth and well-approximated by low-dimensional bases. The computational cost of SEES is $O(\text{basis_dim})$, independent of $|\mathcal{S}|$, which is why it maintains sub-3-second runtimes on 400 states while NFXP takes 7.6 seconds. At higher discount factors ($\beta = 0.9999$) the SEES advantage grows to 30x because NFXP inner iterations scale with $(1-\beta)^{-1}$ while SEES avoids fixed-point iteration entirely. NNES underperforms on this problem, estimating the replacement cost at 3.525 versus 3.052 from NFXP, because the neural value network struggles to fit 400 states from noisy Bellman targets.

6.3 Case Study 3: Nonlinear Rewards

We use the Objectworld environment of Wulfmeier et al. (2016), an 8×8 gridworld with 64 states and 5 actions. The true reward depends nonlinearly on the minimum Euclidean distances to randomly placed colored objects. We generate 200 expert trajectories of length 50 and compare linear MCE-IRL (with continuous distance features), f-IRL with two divergence choices (KL and chi-squared), and behavioral cloning.

Table 5: Case Study 3: Objectworld (8×8, 64 states, nonlinear reward). Policy accuracy is the fraction of states where the learned policy matches the expert.

Estimator	Policy Accuracy (%)	Time (s)
MCE-IRL (linear features)	100.0	415.7
f-IRL (χ^2 divergence)	89.1	23.8
f-IRL (KL divergence)	79.7	22.4
BC (baseline)	—	<1

MCE-IRL with well-designed continuous distance features achieves 100 percent policy accuracy even though the true reward is nonlinear, because the continuous features capture the monotonic relationship between distance and reward. Feature design matters more than estimator sophistication for policy accuracy. f-IRL recovers 79 to 89 percent policy accuracy without any feature engineering, using a fully nonparametric tabular reward. The chi-squared divergence outperforms KL (89 versus 80 percent) due to its stronger penalization of large occupancy mismatches, which explains the remark in Garg et al. (2021) that chi-squared objectives are more readily interpretable than

alternatives. f-IRL runs roughly 18 times faster than MCE-IRL because it avoids the inner soft value iteration loop.

6.4 Real-Data Applications

Table 6: Real-data applications. All estimators produce standard errors, pass pre-estimation diagnostics, and support counterfactual simulation.

Application	Domain	N	Indiv.	States	Estimator	Log-Lik
RDW Scrappage	Vehicle maintenance	2,009,144	170,852	22	NFXP	−2,833
Expedia Search	Recommendation	120,527	8,625	30	NPL	−38,387
KKBox Churn	Subscription	61,750	4,961	12	NPL	−7,655
AHS Housing	Tenure choice	100,000	5,000	54	NFXP	−17,292

The RDW scrappage application uses 2 million observations from the Dutch vehicle registry to estimate the cost of vehicle defects and the implied replacement cost, achieving 99.2 percent prediction accuracy. The Expedia and KKBox applications estimate dynamic models of consumer search and subscription renewal using CCP-NPL, which converges in under 30 seconds on datasets with over 100,000 observations. All four applications pass the pre-estimation diagnostics described in Section 5, with full-rank feature matrices and adequate state coverage.

7 Conclusion

EconIRL provides twelve interchangeable estimators spanning structural econometrics and inverse reinforcement learning, unified by a common `Estimator` protocol and shared post-estimation infrastructure for standard errors, identification diagnostics, and counterfactual simulation. The tabular case study in Section 6 confirms the theoretical equivalence of NFXP, CCP, and MCE-IRL, with all three recovering the same parameters and producing identical policy performance on the Rust (1987) bus engine problem. The scaling study shows that sieve and neural methods extend to high-dimensional state spaces without sacrificing parameter accuracy, maintaining cosine similarity above 0.99 to the true reward vector even as the number of states grows into the thousands. The nonlinear reward study demonstrates where deep IRL adds value over linear models, recovering reward curvature that linear specifications miss entirely.

Several limitations remain. All estimators except TD-CCP require known or consistently estimated transition matrices, which limits applicability in settings where the state transition process is complex or partially observed. The adversarial estimators, particularly AIRL, lack standard errors because the instability of GAN-style minimax training makes the asymptotic distribution of the discriminator parameters difficult to characterize. GLADIUS exhibits systematic bias in the IRL setting due to state-dependent identification constants that leak through asymmetric transitions, overestimating operating costs by approximately 40 percent on the bus engine benchmark regardless of network architecture or training duration.

Future extensions include dynamic games with strategic interaction (Aguirregabiria and Mira, 2002), unobserved heterogeneity via finite mixture and random coefficient models, and continuous-state deep estimators with valid inference guarantees. Integration with automatic differentiation through equilibrium constraints would enable efficient computation of standard errors for the adversarial estimators, addressing the most significant gap in the current inference pipeline. The package is MIT licensed and available at `https://github.com/econirl/econirl`.

References

Karun Adusumilli and Dita Eckardt. Temporal-difference estimation of dynamic discrete choice models. Working Paper, 2025.

Victor Aguirregabiria and Pedro Mira. Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models. *Econometrica*, 70(4):1519–1543, 2002.

Victor Aguirregabiria and Pedro Mira. Dynamic discrete choice structural models: A survey. *Journal of Econometrics*, 156(1):38–67, 2010.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax.

Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. In *International Conference on Learning Representations*, 2018.

Divyansh Garg, Shuvam Chakraborty, Chris Cundy, Jiaming Song, and Stefano Ermon. IQ-Learn: Inverse soft-Q learning for imitation. In *Advances in Neural Information Processing Systems*, volume 35, 2021.

Adam Gleave, Sam Toyer, and and others. imitation: Clean imitation learning implementations. *arXiv preprint arXiv:2211.11972*, 2022.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Fedor Iskhakov, Thomas H. Jorgensen, John Rust, and Bertel Schjerning. Comment on "Constrained optimization approaches to estimation of structural models". *Econometrica*, 84(1):365–370, 2016.

Enoch H. Kang, Hema Yoganarasimhan, and Lalit Jain. An empirical risk minimization approach for offline inverse RL and dynamic discrete choice model. *arXiv preprint arXiv:2502.14131*, 2025.

Patrick Kidger and Cristian Garcia. Equinox: Neural networks in JAX via callable PyTrees and filtered transformations. In *Differentiable Programming Workshop at NeurIPS*, 2021.

Kuno Kim, Shivam Garg, Kirthevasan Shiragur, and Stefano Ermon. Reward identification in inverse reinforcement learning. *Proceedings of the 38th International Conference on Machine Learning*, 2021.

Yao Luo and Yingying Sang. Sieve estimation of dynamic discrete choice models. Working Paper, 2024.

Thierry Magnac and David Thesmar. Identifying dynamic discrete decision processes. *Econometrica*, 70(2):801–816, 2002.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning*, pages 278–287, 1999.

Hoang Nguyen. Neural network estimators for dynamic discrete choice models. Georgetown University Working Paper, 2025.

Tianwei Ni, Harshit Sikchi, Yufei Wang, Tejus Gupta, Lisa Lee, and Ben Eysenbach. f-IRL: Inverse reinforcement learning via state marginal matching. In *Conference on Robot Learning*, 2022.

Dean A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.

Pranjal Rawat. IQ-Learn as penalized conditional log-likelihood: A tabular reinterpretation for the DDC setting. Georgetown University Working Paper, 2026.

Stéphane Ross, Geoffrey Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

John Rust. Optimal replacement of GMC bus engines: An empirical model of Harold Zurcher. *Econometrica*, 55(5):999–1033, 1987.

Che-Lin Su and Kenneth L. Judd. Constrained optimization approaches to estimation of structural models. *Econometrica*, 80(5):2213–2230, 2012.

Markus Wulfmeier, Peter Ondruska, and Ingmar Posner. Maximum entropy deep inverse reinforcement learning. *arXiv preprint arXiv:1507.04888*, 2016.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

A IQ-Learn as Penalized Conditional Log-Likelihood

The IQ-Learn estimator (Garg et al., 2021) parameterizes the Q-function directly and minimizes a chi-squared divergence objective over it, bypassing the need to specify a reward function. In continuous environments where the transition kernel $p(s'|s,a)$ is unknown, the continuation value $\sum_{s'} V(s')\, p(s'|s,a)$ must be approximated from sampled next states, introducing double-sampling bias and requiring careful target-network stabilization. In the tabular DDC setting where transitions are known from the model or consistently estimated from data, this expectation can be computed exactly from the transition matrix. Rawat (2026) shows that the resulting finite-state objective has a clean structural interpretation that connects IQ-Learn to the classical maximum likelihood estimators of structural econometrics.

The chi-squared objective. Let $\{(s_i, a_i)\}_{i=1}^N$ denote the observed state-action pairs. The IQ-Learn chi-squared objective is

$$\mathcal{L}(Q) \;=\; -\frac{1}{N}\sum_{i=1}^{N}\big[Q(s_i,a_i) - V(s_i)\big] \;+\; \frac{1}{4\alpha}\,\frac{1}{N}\sum_{i=1}^{N}\big[\mathrm{td}(s_i,a_i)\big]^2, \tag{5}$$

where $V(s) = \sigma \log \sum_a \exp\{Q(s,a)/\sigma\}$ is the smoothed value function (the log-sum-exp operator), $\mathrm{td}(s,a) = Q(s,a) - \beta \sum_{s'} V(s')\, p(s'|s,a)$ is the temporal difference between the current Q-value and its Bellman backup, and $\alpha > 0$ is a regularization parameter that controls the strength of the Bellman-consistency penalty. The first term rewards Q-functions that assign high advantage to observed actions. The second term penalizes Q-functions that deviate from the Bellman fixed point.

Two identities. Two algebraic identities, proved in Rawat (2026), enable the reinterpretation.

Identity 1 (Proposition 1 of Rawat, 2026). For any function $Q(s,a)$, define $V(s)$ via the log-sum-exp and $\pi(a|s) = \exp\{(Q(s,a) - V(s))/\sigma\}$ via the softmax. Then $Q(s,a) - V(s) = \sigma \log \pi(a|s)$. The advantage function equals the scaled log choice probability under the softmax policy induced by Q.

Identity 2 (Proposition 2 of Rawat, 2026). If Q satisfies the Bellman fixed point $Q = \Lambda_\sigma(Q)$, where Λ_σ denotes the smoothed Bellman operator, then $\mathrm{td}(s,a) = r(s,a)$. The temporal difference equals the flow reward implied by Q through the inverse Bellman operator.

The equivalence. Substituting these identities into equation (5) yields the main result of Rawat (2026), stated here without proof.

$$\boxed{\min_Q \mathcal{L}(Q) \iff \max_Q \underbrace{\frac{1}{N}\sum_{i=1}^{N} \log \pi(a_i \mid s_i)}_{\text{conditional log-likelihood}} - \frac{1}{4\alpha\sigma}\sum_{i=1}^{N} \hat{r}_i^2} \tag{6}$$

The term $\mathrm{LL}(Q) = \sum_i \log \pi(a_i|s_i)$ is the conditional log-likelihood of observed actions under the softmax policy induced by Q, which is exactly the standard multinomial logit likelihood with Q-values playing the role of choice-specific utilities. The term $\hat{r}_i = \mathrm{td}(s_i, a_i)$ is the Bellman-implied reward at the i-th observation, so the penalty $\sum_i \hat{r}_i^2$ is an ℓ^2 norm on the magnitude of the implied reward at expert-visited state-action pairs. Minimizing the IQ-Learn objective is therefore equivalent to maximizing the fit to observed choices while penalizing the complexity of the reward function that rationalizes them.

Connection to NFXP. In NFXP (Rust, 1987; Iskhakov et al., 2016), the Bellman equation $Q = \Lambda_\sigma(Q)$ is imposed as a hard constraint. An inner loop solves the fixed point to machine precision before the outer loop evaluates the conditional log-likelihood and takes a gradient step in the structural parameters θ. In IQ-Learn, the Bellman equation enters as a soft constraint through the ℓ^2 penalty on temporal differences. The regularization parameter α governs the tradeoff between data fit and structural consistency. As $\alpha \to 0$ the penalty dominates and forces $\mathrm{td}(s,a) \to 0$ at observed state-action pairs, so IQ-Learn converges to the Bellman-constrained MLE that NFXP solves exactly. As $\alpha \to \infty$ the penalty vanishes and the estimator reduces to an unconstrained conditional logit that maximizes the likelihood of observed choices with no structural content from the dynamic model. The IQ-Learn regularization path therefore traces a continuous family of estimators between the unconstrained logit and the fully structural NFXP.

Connection to CCP estimation. Both CCP estimation (Hotz and Miller, 1993; Aguirregabiria and Mira, 2002) and IQ-Learn avoid the inner fixed-point loop that makes NFXP computationally expensive at high discount factors. CCP methods estimate choice probabilities nonparametrically in a first stage, use the Hotz-Miller inversion to recover continuation values from these probabilities, and then estimate the structural parameters θ in a second-stage conditional likelihood. IQ-Learn estimates the Q-function directly and recovers the implied reward via the inverse Bellman operator $r(s,a) = Q(s,a) - \beta \sum_{s'} V(s')\, p(s'|s,a)$. Both approaches replace the nested computation with a direct estimation step, but they differ in what is estimated in the first stage. CCP methods estimate the policy $\pi(a|s)$ and work backward to value functions, while IQ-Learn estimates $Q(s,a)$ and reads off the policy as a byproduct through the softmax.

Connection to GLADIUS. The ERM-IRL estimator of Kang et al. (2025) shares the same high-level structure as the tabular IQ-Learn rewrite in equation (6). Both objectives combine a conditional log-likelihood term with a Bellman-consistency penalty, and both avoid the inner fixed-point loop. The key difference lies in how the continuation value $\mathbb{E}[V(s') \mid s, a]$ is handled. Kang et al. (2025) work in a model-free setting where $p(s'|s, a)$ is unknown and introduce a dual variable $\zeta(s, a)$ that approximates the conditional expectation, turning the problem into a min-max optimization with provable convergence guarantees under their ERM framework. In the tabular DDC setting with known transitions, this expectation is computed exactly from the transition matrix, which eliminates the need for the dual variable and its associated double-sampling bias. The tabular IQ-Learn equivalence therefore reveals the simplest member of a family of penalized-likelihood estimators that also includes GLADIUS, with each member distinguished by its treatment of the transition kernel.

B Anchored AIRL with Unobserved Heterogeneity

This appendix describes the Anchored AIRL (AAIRL) algorithm, which extends the AIRL estimator of Fu et al. (2018) with latent user segments and anchor constraints for point identification of rewards following Lee, Sudhir, and Wang (2026). The algorithm recovers segment-specific reward functions $r^*(s, a, z)$ and policies $\pi^*(a \mid s, z)$ from observed trajectories under the maximum causal entropy framework.

Setup. Each user i belongs to a latent segment $z \in \{1, \ldots, K\}$ with K determined by the researcher. Segment membership is shared across all series or episodes consumed by the same user, providing cross-episode identification. The objective is

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t, z) + \mathcal{H}(\pi(\cdot \mid s_t, z))\big)\right].$$

Networks. Four neural networks parameterize the model.

Component	Parameterizes	Architecture
$g_\theta(s, a, z)$	Immediate reward	2-layer MLP, Leaky ReLU
$h_\phi(s, z)$	Shaping potential $\approx V(s)$	2-layer MLP, Leaky ReLU
$\pi_\omega(a \mid s, z)$	Policy (CCPs)	2-layer MLP, Leaky ReLU
$q_\psi(z \mid \tau)$	Posterior (segment assignment)	Bi-LSTM + MLP

Discriminator structure. The discriminator embeds a Bellman-consistent decomposition

$$f_{\theta,\phi}(s, a, s', z) = g_\theta(s, a, z) + \gamma\, h_\phi(s', z) - h_\phi(s, z), \tag{7}$$

$$D_{\theta,\phi}(s, a, s', z) = \frac{\exp\big(f_{\theta,\phi}\big)}{\exp\big(f_{\theta,\phi}\big) + \pi_\omega(a \mid s, z)}. \tag{8}$$

Anchor constraints. The anchor action (exit) is constrained to have zero reward $g_\theta(s, \text{exit}, z) = 0$ for all s, z, and the absorbing state has zero value $h_\phi(s_{\text{abs}}, z) = 0$. These constraints resolve the reward shaping ambiguity discussed in Section 2, making the decomposition unique at convergence so that $g_\theta(s, a, z) = r^*(s, a, z)$ and $h_\phi(s, z) = V^*(s, z)$.

Algorithm. The five steps iterate until convergence.

Initialize parameters $\theta_0, \phi_0, \omega_0, \psi_0$

↓

S1. Simulate trajectories. For each segment z, roll out $\pi_\omega(a \mid s, z)$ to generate synthetic transitions $\mathcal{D}_{\text{sim}}$.

↓

S2. Discriminator update (updates θ, ϕ). Compute f and D via Equations (7)–(8). Minimize $\mathcal{L}_{\text{disc}} = -\frac{1}{|\mathcal{D}_{\text{obs}}|} \sum \log D - \frac{1}{|\mathcal{D}_{\text{sim}}|} \sum \log(1 - D)$.

↓

S3. Posterior update (updates ψ). Compute $q_\psi(z \mid \tau_i)$ for each trajectory. Minimize $\mathcal{L}_{\text{classify}} + \lambda_{\text{CR}} \cdot \mathcal{L}_{\text{consistency}}$ (same user across series gets similar z).

↓

S4. Reward assignment (no parameter updates). For each simulated (s, a, s'): $\tilde{r} = f_{\theta,\phi}(s, a, s', z) - \log \pi_\omega(a \mid s, z) + \lambda_{\text{MI}} \log q_\psi(z \mid \tau)$.

↓

S5. Policy update (updates ω). For each segment z, update ω via PPO on $\max_{\pi_\omega} \mathbb{E}[\sum_t \gamma^t \tilde{r}]$.

↓

Check: $\max_{s,a,z} |\pi^{\text{new}} - \pi^{\text{old}}| < \varepsilon$? Stop. Else repeat from S1.

repeat

At convergence. When $D = 1/2$ everywhere, the discriminator cannot distinguish observed from simulated transitions, and $f_{\theta,\phi} = \log \pi^*(a \mid s, z) = A^*(s, a, z)$. With the anchor constraints enforced, the decomposition is unique and g_θ recovers the true segment-specific reward function.